

Devoir1 : INFO 4307

Oumar Leyti Ndiaye A00228981

Adresse IP de chaque machine :

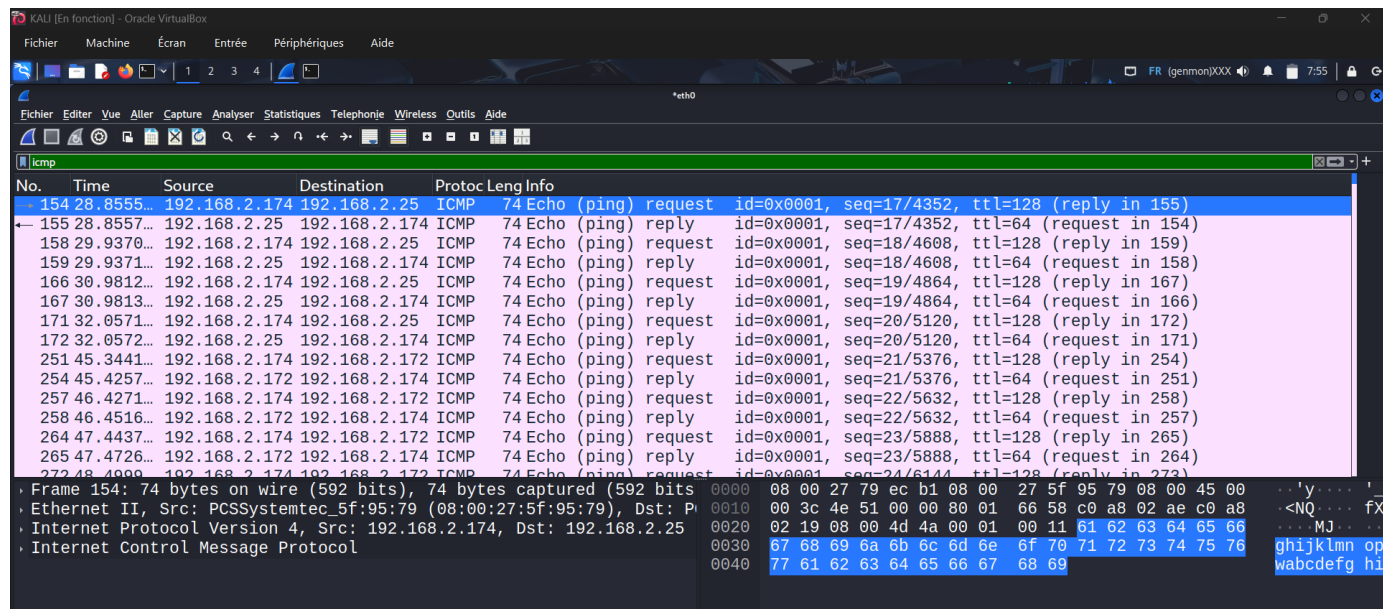
Windows 10 : 192.168.2.174

Kali Linux : 192.168.2.25

Metasploitable 2 : 192.168.2.172

Tâche 2 : interception d'un ping (35 points)

7. Décrire et analyser certains détails d'un paquet ICMP



Voici ci-dessus les résultats après analyse des paquet réseaux avec Wireshark sur l'interface eth0 en appliquant le filtre sur les paquets ICMP.

On observe des paquets ICMP échangés entre la machine 192.168.2.174 (windows10) et la machine 192.168.2.25 (Kali Linux).

No.	Time	Source	Destination	Protoc	Leng	Info
154	28.8555...	192.168.2.174	192.168.2.25	ICMP	74	Echo (ping) request id=0x0001, seq=17/4352, ttl=128 (reply in 155)
155	28.8557...	192.168.2.25	192.168.2.174	ICMP	74	Echo (ping) reply id=0x0001, seq=17/4352, ttl=64 (request in 154)
158	29.9370...	192.168.2.174	192.168.2.25	ICMP	74	Echo (ping) request id=0x0001, seq=18/4608, ttl=128 (reply in 159)

Dans les informations du paquet, on observe plusieurs champs importants :

- Protocol: ICMP
- Length: 74 bytes
- Type: Echo Reply

- Sequence number : 17
- TTL : 64

Le champ **TTL** (Time To Live) indique le nombre maximal de sauts réseau qu'un paquet peut effectuer avant d'être supprimé. Dans ce cas, la valeur 64 est typique des systèmes Linux.

On observe également un paquet **Echo Reply** envoyé en réponse par l'adresse 192.168.2.25, confirmant que la communication entre les deux machines est fonctionnelle. (Exemple : les paquets 154 & 155)

8. Quels sont les différents types de paquets ICMP que vous avez observés dans la capture ?

Dans la capture, nous avons deux types de paquets ICMP :

- Type 8 - Echo Request : Ce paquet est envoyé par la machine 192.168.2.174 (Windows 10) c'est-à-dire la machine qui lance la commande ping pour tester la connectivité avec la machine 192.168.2.25 (Kali). Le TTL est de 128, ce qui trahit une origine Windows.
- Type 0 - Echo Reply : Envoyé en retour par Kali (192.168.2.25) et Metasploitable 2 (192.168.2.172) vers Windows 10. TTL = 64, typique de Linux, cohérent avec ces deux machines.

9. Quelles sont les informations importantes contenues dans l'en-tête d'un paquet ICMP ?

```

272.48.4000 192.168.2.174 192.168.2.172 ICMP 74 Echo (ping) request
> Frame 155: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0
> Ethernet II, Src: PCSSystemtec_79:ec:b1 (08:00:27:79:ec:b1), Dst: PCSSystemtec_79:ec:b1 (08:00:27:79:ec:b1)
> Internet Protocol Version 4, Src: 192.168.2.25, Dst: 192.168.2.174
> Internet Control Message Protocol
  Type: 0 (Echo (ping) reply)
  Code: 0
  Checksum: 0x554a [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 17 (0x0011)
  Sequence Number (LE): 4352 (0x1100)
  [Request frame: 154]
  [Response time: 0,138 ms]
  Data (32 bytes)

```

Dans l'en-tête, nous avons les informations suivantes :

- **Type** : Indique le type du message ICMP (8 = request et 0 = reply)
- **Code** : Fournit des informations supplémentaires sur le type de message ICMP. Vaut 0 pour les Echo (request/ reply) et prend d'autres valeurs pour les messages d'erreur ICMP comme **Destination Unreachable**.
- **Checksum** : Permet de vérifier l'intégrité des données du paquet. Statut Good. Garantit l'intégrité du paquet entre l'émetteur et le récepteur.

- **Identifier** : Permet d'associer les replies aux requests d'une même session, utile si plusieurs pings tournent simultanément.
- **Sequence Number** : Permet de numéroter les paquets envoyés afin de suivre leur ordre.
- **Data** : C'est le contenu envoyé avec le ping.

10. Comment pouvez-vous identifier les anomalies ou les comportements suspects dans la capture Wireshark, en particulier en ce qui concerne les paquets ICMP ?

En observant la capture (ci-dessus), plusieurs indicateurs permettent de repérer un comportement suspect :

1. **Rupture de séquence** : Les numéros s'incrémentent normalement (17→18→19→20...). Un numéro sauté indiquerait une perte de paquet ; un numéro répété indiquerait une duplication ou une injection.
2. **Asymétrie request/reply** : Dans la capture, chaque request a son reply. Des requests sans reply indiqueraient un hôte filtré ou éteint. Des replies sans requests correspondants seraient très suspects — signe possible de paquets forgés ou de spoofing.
3. **Volume anormal** : Les pings arrivent à intervalles d'environ 1 seconde (timestamps 28.8, 29.9, 30.9...), ce qui est normal. Un flood ICMP produirait des centaines de paquets par seconde avec des timestamps quasi identiques.
4. **TTL incohérent** : TTL=128 pour Windows 10 et TTL=64 pour Kali, ce qui est parfaitement cohérent. Un TTL inhabituel pour une machine connue signalerait du spoofing d'adresse IP.
5. **Taille anormale** : tous les paquets font 74 bytes (standard). Un paquet ICMP anormalement grand pourrait indiquer une tentative de fragmentation malveillante.

11. Où se trouve le champ Checksum dans l'en-tête ICMP et quel est son rôle dans le fonctionnement du protocole ?

Le **champ Checksum** se trouve dans l'en-tête ICMP, immédiatement après Type et Code. (Voir l'image de la question 9)

Son **rôle** est de **vérifier l'intégrité du paquet pendant la transmission sur le réseau**. Lorsqu'un paquet est envoyé, une valeur de checksum est calculée à partir des données du paquet. À la réception, cette valeur est recalculée et comparée avec celle contenue dans le paquet. Si les deux valeurs correspondent, le paquet est considéré comme valide, d'où la mention : **'Correct'**.

12. Comment Wireshark indique-t-il si le checksum est valide, incorrect ou non vérifié ?

Dans Wireshark, l'état du checksum est indiqué dans les détails du paquet ICMP.

Wireshark peut afficher :

- **Good (notre capture)** : checksum valide, paquet intègre
- **Bad** : valeur incorrecte, paquet corrompu ou modifié
- **Unverified** : vérification non effectuée, souvent dû au checksum offloading où la carte réseau calcule le checksum après la capture de Wireshark

13. Pourquoi la validation du checksum est-elle importante en sécurité réseau et que peut indiquer un checksum incorrect lors de l'analyse d'un trafic ICMP ?

La validation du checksum est importante car elle permet de vérifier que les paquets n'ont pas été altérés pendant leur transmission.

Un Cheksum incorrect peut indiquer :

- **Manipulation man-in-the-middle** : Un attaquant qui modifie un paquet en transit doit recalculer le checksum. Une erreur de calcul le trahit immédiatement dans Wireshark.
- **Injection de paquets forgés** : Un outil qui génère de faux paquets ICMP peut produire des checksums incorrects s'il est mal configuré, révélant ainsi une activité malveillante sur le réseau.
- **Corruption réseau** : Sur un réseau défaillant, les checksums incorrects apparaissent de façon aléatoire et touchent plusieurs protocoles simultanément, pas seulement ICMP.

14. Expliquez dans quels cas cela peut correspondre à une simple erreur technique et dans quels cas cela peut révéler un comportement potentiellement suspect.

Un checksum incorrect peut parfois être dû à une simple erreur technique, par exemple :

- Un problème de transmission
- Une surcharge du réseau
- Certaines optimisations matérielles de la carte réseau

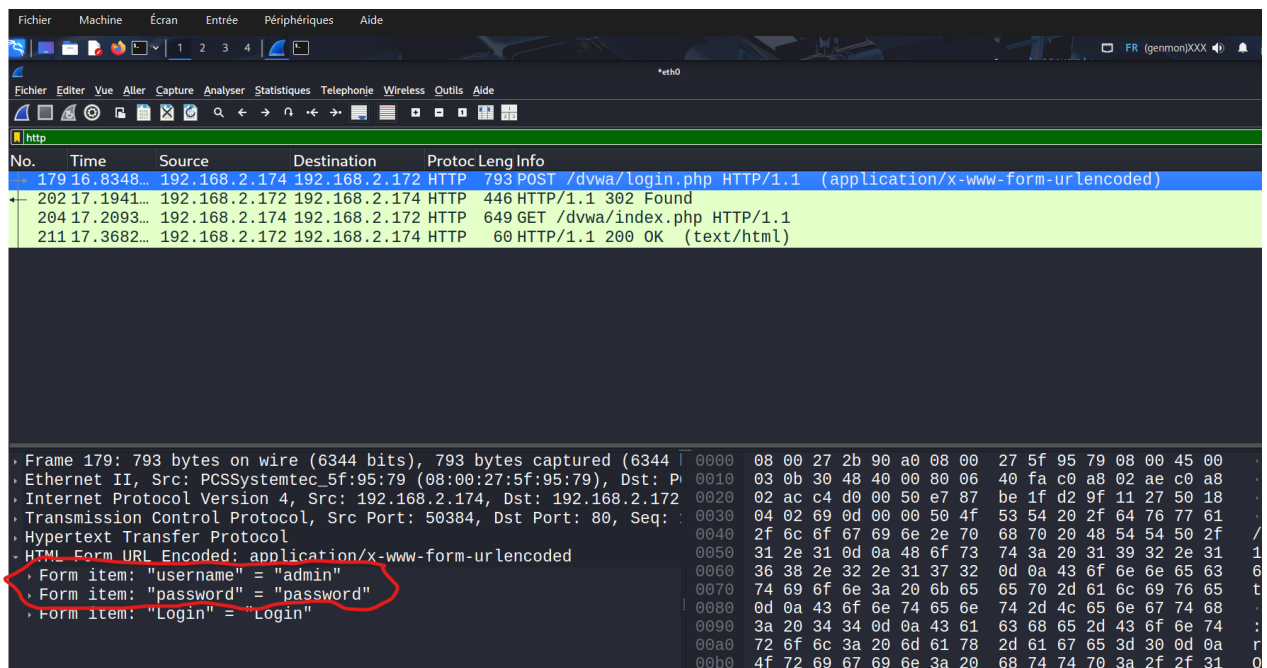
Cependant, si un grand nombre de paquets présentent un checksum incorrect ou si les paquets ICMP sont envoyés de manière anormalement fréquente, cela peut indiquer une activité malveillante comme :

- Une attaque de déni de service
- Une tentative de scan réseau
- Une manipulation du trafic réseau.

Dans la capture observée, les paquets ICMP semblent correspondre à une utilisation normale de la commande **ping** entre les machines Kali et Windows, sans indication d'activité suspecte.

Tâche 3 : Récupération du login et du mot de passe sur Wireshark (20 points)

6. Arrêter la capture, analyser les paquets sur Wireshark et récupérer le nom d'utilisateur et le mot de passe.



Lors de l'analyse du trafic HTTP capturé avec Wireshark, une requête POST vers la page /dvwa/login.php a été observée. Cette requête correspond à l'envoi du formulaire de connexion au site DVWA. Dans les détails du paquet, la section HTML Form URL Encoded contient les paramètres transmis au serveur. Les identifiants de connexion apparaissent clairement :

Username = admin

Password = password

Ces informations sont visibles en clair dans la capture réseau car le site utilise le protocole HTTP, qui ne chiffre pas les données transmises entre le client et le serveur.

Cette situation démontre qu'un attaquant présent sur le réseau peut intercepter des informations sensibles telles que des identifiants de connexion en analysant le trafic réseau avec un outil comme Wireshark.

7. Quels types de paquets sont généralement associés à une tentative de connexion à un service ou une application ?

Plusieurs types de paquets sont impliqués dans une tentative de connexion à une application web :

- **TCP** : avant tout échange HTTP, une connexion TCP est établie entre Windows 10 et Metasploitable via certains paquets. C'est la fondation de la communication.
- **HTTP POST** : c'est le paquet critique, visible ici en n°179. Le navigateur envoie les données du formulaire de connexion (username, password) dans le corps de la requête HTTP. Contrairement au GET, les données ne sont pas dans l'URL mais dans le corps, ce qui donne une fausse impression de sécurité — elles restent totalement visibles dans Wireshark.

- **HTTP 302 Found** : réponse du serveur indiquant une redirection après authentification réussie.
- **HTTP GET** : le navigateur suit la redirection vers index.php.
- **HTTP 200 OK** : confirmation que la page protégée est accessible, donc que la connexion a réussi.

8. Comment pouvez-vous identifier les paquets contenant des informations d'identification, comme le nom d'utilisateur et le mot de passe, dans la capture Wireshark ?

Pour identifier les paquets contenant des identifiants dans Wireshark, plusieurs méthodes peuvent être utilisées.

- **Utiliser un filtre http** : Cela permet d'afficher uniquement les paquets http.
- **Rechercher les requêtes POST** : Les identifiants sont généralement envoyés dans les requêtes POST, car elles transmettent les données des formulaires.
- **Examiner les données du formulaire** : Dans les détails du paquet, Wireshark affiche la section *HTML Form URL Encoded*.

9. Quelles sont les mesures de prévention que vous recommanderiez pour réduire les risques associés à la récupération non autorisée de login et de mot de passe à l'aide de Wireshark ?

Pour éviter qu'un attaquant puisse intercepter les identifiants avec un outil d'analyse réseau comme Wireshark, plusieurs mesures de sécurité peuvent être mises en place.

La mesure la plus importante est d'utiliser HTTPS au lieu de HTTP. Car elle utilise le protocole TLS (Transport Layer Security) qui chiffre les communications entre le client et le serveur. Ainsi, les identifiants ne peuvent pas être lus dans les paquets réseau.

Ensuite, Il est important d'éviter les réseaux publics non sécurisés (Wi-Fi public), car un attaquant sur le même réseau peut facilement capturer le trafic.

De plus, il faut mettre en place des mécanismes de sécurité supplémentaires comme par exemple : authentification à deux facteurs (2FA), politiques de mots de passe forts, détection d'activités suspectes.

Tâche 4 : Autre outil - Tcpdump (25 points)

a-Voir les interfaces disponibles : tcpdump -D

```
Session Actions Éditer Vue Aide

(leyti002@kali)-[~]
$ tcpdump -D
1.eth0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
5.nflog (Linux netfilter log (NFLOG) interface) [none]
6.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
7.dbus-system (D-Bus system bus) [none]
8.dbus-session (D-Bus session bus) [none]

(leyti002@kali)-[~]
$
```

b- Capturer les paquets depuis une interface spécifique (eth0) : tcpdump -i eth0

```
(leyti002@kali)-[~]
$ sudo tcpdump -i eth0
[sudo] Mot de passe de leyti002 :
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
23:41:50.701477 0e:ac:8a:a7:a2:5f (oui Unknown) > 01:80:c2:00:00:00 (oui Unknown), ethertype Unknown (0x2f00), length 61:
0x0000: aaaa 0300 0000 0027 4242 0300 0002 023e .....BB....>
0x0010: 8000 0cac 8aa7 a25a 0000 0000 8000 0cac .....Z.....
0x0020: 8aa7 a25a 8004 0000 1400 0200 0f00 00 ...Z.....
23:41:51.728861 IP 104.18.39.21.https > Leyti.52625: Flags [P.], seq 105830129:105830153, ack 596009255, win 16, length 24
23:41:51.728862 IP Leyti.52625 > 104.18.39.21.https: Flags [P.], seq 1:29, ack 24, win 254, length 28
23:41:51.744200 IP 104.18.39.21.https > Leyti.52625: Flags [.], ack 29, win 16, length 0
23:41:51.815930 ARP, Request who-has _gateway tell kali, length 28
23:41:51.827247 ARP, Reply _gateway is-at 0e:ac:8a:a7:a2:5a (oui Unknown), length 46
23:41:51.827255 IP kali.46830 > _gateway.domain: 21815+ PTR? 62.2.168.192.in-addr.arpa. (43)
23:41:51.829853 IP _gateway.domain > kali.46830: 21815* 1/0/0 PTR Leyti. (62)
23:41:51.830350 IP kali.51696 > _gateway.domain: 46500+ PTR? 21.39.18.104.in-addr.arpa. (43)
23:41:51.838913 IP _gateway.domain > kali.51696: 46500 NXDomain 0/1/0 (105)
23:41:51.934129 IP 172.212.135.2.https > Leyti.64300: Flags [P.], seq 3312043738:3312043842, ack 2087064927, win 2860, length 104
23:41:51.945014 ARP, Request who-has DESKTOP-SQND070 tell _gateway, length 46
23:41:51.945524 ARP, Reply DESKTOP-SQND070 is-at 08:00:27:5f:95:79 (oui Unknown), length 46
23:41:51.974127 IP Leyti.64300 > 172.212.135.2.https: Flags [.], ack 104, win 255, length 0
23:41:52.019961 IP kali.59021 > _gateway.domain: 25883+ PTR? 2.135.212.172.in-addr.arpa. (44)
23:41:52.344275 IP _gateway.domain > kali.59021: 25883 NXDomain 0/1/0 (130)
23:41:52.344829 IP kali.54555 > _gateway.domain: 36284+ PTR? 174.2.168.192.in-addr.arpa. (44)
23:41:52.347858 IP _gateway.domain > kali.54555: 36284* 1/0/0 PTR DESKTOP-SQND070. (73)
23:41:52.547684 ARP, Request who-has Leyti tell _gateway, length 46
23:41:52.547697 ARP, Reply Leyti is-at c0:35:32:eb:8f:5b (oui Unknown), length 46
23:41:52.645691 0e:ac:8a:a7:a2:5f (oui Unknown) > 01:80:c2:00:00:00 (oui Unknown), ethertype Unknown (0x2f00), length 61:
```

c- Capturer un nombre exact de paquets (5) : tcpdump -c 5 -i eth0

```
(leyti002@kali)-[~]
$ sudo tcpdump -c 5 -i eth0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
23:43:56.901281 IP Fibe-TV-Box-FibeTV.mdns > mdns.mcast.net.mdns: 0 A (QM)? 6cb02a04-c205-4fcc-b657-5ad3b04ca290.local. (60)
23:43:56.901282 IP6 fe80::b419:c3a9:8f0d:d7e1.mdns > ff02::fb.mdns: 0 A (QM)? 6cb02a04-c205-4fcc-b657-5ad3b04ca290.local. (60)
23:43:56.902646 IP Fibe-TV-Box-FibeTV.mdns > mdns.mcast.net.mdns: 0*- [0q] 1/0/0 (Cache flush) A 192.168.2.10 (70)
23:43:57.002711 IP Fibe-TV-Box-FibeTV.mdns > mdns.mcast.net.mdns: 0 PTR (QM)? _googlecast._tcp.local. (40)
23:43:57.002713 IP6 fe80::b419:c3a9:8f0d:d7e1.mdns > ff02::fb.mdns: 0 PTR (QM)? _googlecast._tcp.local. (40)
5 packets captured
15 packets received by filter
0 packets dropped by kernel
```

d- Capturer les paquets TCP uniquement : tcpdump -i eth0 tcp

```

(leyti002@kali)~$ sudo tcpdump -i eth0 tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
23:45:39.480735 IP DESKTOP-SQND070.49937 > Fibe-TV-Box-FibeTV.8009: Flags [P.], seq 4238990887:4238990997, ack 948805994, win 1022, length 110
23:45:39.498396 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49937: Flags [P.], seq 1:111, ack 110, win 302, length 110
23:45:39.570404 IP DESKTOP-SQND070.49937 > Fibe-TV-Box-FibeTV.8009: Flags [.] , ack 111, win 1022, length 0
23:45:39.870334 IP DESKTOP-SQND070.49939 > Fibe-TV-Box-Living-Room-TV.8009: Flags [P.], seq 3060471776:3060471886, ack 2988262412, win 1024, length 110
23:45:39.886734 IP Fibe-TV-Box-Living-Room-TV.8009 > DESKTOP-SQND070.49939: Flags [P.], seq 1:111, ack 110, win 325, length 110
23:45:39.929938 IP DESKTOP-SQND070.49939 > Fibe-TV-Box-Living-Room-TV.8009: Flags [.] , ack 111, win 1024, length 0
23:45:41.042878 IP Leytl.58741 > 52.123.251.159.https: Flags [.] , seq 1405952642:1405952643, ack 22317083, win 1023, length 1
23:45:41.100083 IP 52.123.251.159.https > Leytl.58741: Flags [.] , ack 1, win 255, options [nop,nop,sack 1 {0:1}], length 0
23:45:41.655786 IP Leytl.60687 > 52.123.251.155.https: Flags [.] , seq 3195830222:3195830233, ack 3811619175, win 511, length 1
23:45:41.714516 IP 52.123.251.155.https > Leytl.60687: Flags [.] , ack 1, win 250, options [nop,nop,sack 1 {0:1}], length 0
23:45:43.025697 IP Leytl.55331 > whatsapp-cdn-shv-02-lga3.fbcnd.net.xmpp-client: Flags [P.], seq 3636878990:3636879059, ack 2923803046, win 255, length 69
23:45:43.118443 IP whatsapp-cdn-shv-02-lga3.fbcnd.net.xmpp-client > Leytl.55331: Flags [.] , ack 69, win 300, length 0
23:45:43.123885 IP whatsapp-cdn-shv-02-lga3.fbcnd.net.xmpp-client > Leytl.55331: Flags [P.], seq 1:72, ack 69, win 300, length 71
23:45:43.163711 IP Leytl.55331 > whatsapp-cdn-shv-02-lga3.fbcnd.net.xmpp-client: Flags [.] , ack 72, win 255, length 0
23:45:44.504126 IP DESKTOP-SQND070.49937 > Fibe-TV-Box-FibeTV.8009: Flags [P.], seq 110:220, ack 111, win 1022, length 110
23:45:44.520365 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49937: Flags [P.], seq 111:221, ack 220, win 302, length 110
23:45:44.570708 IP DESKTOP-SQND070.49937 > Fibe-TV-Box-FibeTV.8009: Flags [.] , ack 221, win 1021, length 0
23:45:44.913721 IP DESKTOP-SQND070.49939 > Fibe-TV-Box-Living-Room-TV.8009: Flags [P.], seq 110:220, ack 111, win 1024, length 110
23:45:44.929934 IP Fibe-TV-Box-Living-Room-TV.8009 > DESKTOP-SQND070.49939: Flags [P.], seq 111:221, ack 220, win 325, length 110
23:45:44.983766 IP DESKTOP-SQND070.49939 > Fibe-TV-Box-Living-Room-TV.8009: Flags [.] , ack 221, win 1023, length 0
23:45:47.983470 IP Leytl.64300 > 172.212.135.2.https: Flags [P.], seq 2087065588:2087065616, ack 3312045807, win 253, length 28
23:45:48.035596 IP 172.212.135.2.https > Leytl.64300: Flags [.] , ack 28, win 2860, length 0
23:45:49.365950 IP 172.172.255.216.https > Leytl.62591: Flags [P.], seq 3919224251:3919224724, ack 4030525221, win 7961, length 473
23:45:49.366754 IP Leytl.62591 > 172.172.255.216.https: Flags [P.], seq 1:79, ack 473, win 250, length 78
23:45:49.385785 IP Leytl.53700 > 52.104.17.53.https: Flags [F.], seq 3185952087, ack 30471866, win 255, length 0
23:45:49.386728 IP Leytl.55332 > 52.104.17.53.https: Flags [S.], seq 3510542369, win 65535, options [mss 1460,nop,wscale 8,nop,nop,sackOK], length 0
23:45:49.478227 IP 172.172.255.216.https > Leytl.62591: Flags [.] , ack 79, win 7883, length 0
23:45:49.478229 IP 52.104.17.53.https > Leytl.55332: Flags [S.], seq 1517321337, ack 3510942370, win 65535, options [mss 1460,nop,wscale 8,nop,nop,sackOK], length 0

```

e- Capturer les paquets d'un port spécifié (80) : tcpdump -i eth0 port 80 (pour cela tentez une recherche sur le navigateur)

```

(leyti002@kali)~$ sudo tcpdump -i eth0 port 80
[sudo] Mot de passe de leyti002 :
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
02:53:04.718182 IP kali.36080 > lcigaa-az-in-f3.1e100.net.http: Flags [.] , ack 3304861861, win 508, options [nop,nop,TS val 1861520584 ecr 1681136930], length 0
02:53:04.819566 IP lcigaa-az-in-f3.1e100.net.http > kali.36080: Flags [.] , ack 1, win 1050, options [nop,nop,TS val 1681147062 ecr 1061510532], length 0
02:53:13.698175 IP kali.44484 > 82.221.107.34.bc.googleusercontent.com.http: Flags [.] , ack 996516795, win 501, options [nop,nop,TS val 331140455 ecr 2678004718], length 0
02:53:13.729808 IP 82.221.107.34.bc.googleusercontent.com.http > kali.44484: Flags [.] , ack 1, win 1050, options [nop,nop,TS val 2678014954 ecr 331120159], length 0
02:53:13.953045 IP kali.55984 > 104.18.38.233.http: Flags [.] , ack 2329788492, win 501, options [nop,nop,TS val 2309650757 ecr 81264693], length 0
02:53:13.954310 IP kali.45550 > lcigaa-az-in-f3.1e100.net.http: Flags [.] , ack 1110986498, win 508, options [nop,nop,TS val 1861529800 ecr 1681146030], length 0
02:53:13.980893 IP 104.18.38.233.http > kali.55984: Flags [.] , ack 1, win 16, options [nop,nop,TS val 81306607 ecr 2309630635], length 0
02:53:13.981815 IP lcigaa-az-in-f3.1e100.net.http > kali.45550: Flags [.] , ack 1, win 1050, options [nop,nop,TS val 1681156273 ecr 1061499357], length 0
02:53:14.214534 IP kali.55996 > 104.18.38.233.http: Flags [.] , ack 1830621289, win 508, options [nop,nop,TS val 2309651018 ecr 1813544400], length 0
02:53:14.237775 IP 104.18.38.233.http > kali.55996: Flags [.] , ack 1, win 16, options [nop,nop,TS val 1813558634 ecr 2309630659], length 0
02:53:14.450902 IP kali.36926 > 192.64.151.235.http: Flags [S.], seq 581479736, win 64240, options [mss 1460,sackOK,TS val 557781020 ecr 0,nop,wscale 7], length 0
02:53:14.451429 IP kali.36938 > 192.64.151.235.http: Flags [S.], seq 1461251365, win 64240, options [mss 1460,sackOK,TS val 557781020 ecr 0,nop,wscale 7], length 0
02:53:14.481131 IP 192.64.151.235.http > kali.36926: Flags [S.], seq 1880167641, ack 581479737, win 65160, options [mss 1460,sackOK,TS val 1817421994 ecr 557781020,nop,wscale 7], length 0
02:53:14.481132 IP 192.64.151.235.http > kali.36938: Flags [S.], seq 1231786155, ack 1461251366, win 65160, options [mss 1460,sackOK,TS val 1817421994 ecr 557781020,nop,wscale 7], length 0
02:53:14.481647 IP kali.36926 > 192.64.151.235.http: Flags [.] , ack 1, win 502, options [nop,nop,TS val 557781050 ecr 1817421994], length 0
02:53:14.481901 IP kali.36938 > 192.64.151.235.http: Flags [.] , ack 1, win 502, options [nop,nop,TS val 557781051 ecr 1817421994], length 0
02:53:14.482474 IP kali.36926 > 192.64.151.235.http: Flags [P.], seq 1:331, ack 1, win 502, options [nop,nop,TS val 557781051 ecr 1817421994], length 330: HTTP: GET / HTTP/1.1
02:53:14.550601 IP 192.64.151.235.http > kali.36926: Flags [.] , ack 331, win 507, options [nop,nop,TS val 1817422025 ecr 557781051], length 0
02:53:14.550603 IP 192.64.151.235.http > kali.36926: Flags [P.], seq 1:351, ack 331, win 507, options [nop,nop,TS val 1817422025 ecr 557781051], length 350: HTTP: HTTP/1.1 302 Moved Temporarily
02:53:14.550661 IP kali.36926 > 192.64.151.235.http: Flags [.] , ack 351, win 501, options [nop,nop,TS val 557781120 ecr 1817422025], length 0
02:53:14.981850 IP kali.36080 > lcigaa-az-in-f3.1e100.net.http: Flags [.] , ack 1, win 508, options [nop,nop,TS val 1861530828 ecr 1681147062], length 0
02:53:15.013160 IP lcigaa-az-in-f3.1e100.net.http > kali.36080: Flags [.] , ack 1, win 1050, options [nop,nop,TS val 1681157381 ecr 1061510532], length 0
02:53:19.660405 IP kali.36938 > 192.64.151.235.http: Flags [F.], seq 1, ack 1, win 502, options [nop,nop,TS val 557786233 ecr 1817421994], length 0
02:53:19.778572 IP 192.64.151.235.http > kali.36938: Flags [F.], seq 1, ack 2, win 510, options [nop,nop,TS val 1817422719 ecr 557786233], length 0
02:53:19.778620 IP kali.36938 > 192.64.151.235.http: Flags [.] , ack 2, win 502, options [nop,nop,TS val 557786348 ecr 1817422719], length 0
02:53:23.946236 IP kali.44484 > 82.221.107.34.bc.googleusercontent.com.http: Flags [.] , ack 1, win 501, options [nop,nop,TS val 331150703 ecr 2678014954], length 0
02:53:23.972664 IP 82.221.107.34.bc.googleusercontent.com.http > kali.44484: Flags [.] , ack 1, win 1050, options [nop,nop,TS val 2678025196 ecr 331120159], length 0
02:53:24.200813 IP kali.45550 > lcigaa-az-in-f3.1e100.net.http: Flags [.] , ack 1, win 508, options [nop,nop,TS val 1061540046 ecr 1681156273], length 0
02:53:24.200838 IP kali.55984 > 104.18.38.233.http: Flags [.] , ack 1, win 501, options [nop,nop,TS val 230966104 ecr 81306607], length 0
02:53:24.223115 IP 104.18.38.233.http > kali.55984: Flags [.] , ack 1, win 16, options [nop,nop,TS val 81316848 ecr 2309630635], length 0
02:53:24.229544 IP lcigaa-az-in-f3.1e100.net.http > kali.45550: Flags [.] , ack 1, win 1050, options [nop,nop,TS val 1681166515 ecr 1061499357], length 0
02:53:24.456436 IP kali.55996 > 104.18.38.233.http: Flags [.] , ack 1, win 508, options [nop,nop,TS val 2309661259 ecr 1813558634], length 0
02:53:24.479207 IP 104.18.38.233.http > kali.55996: Flags [.] , ack 1, win 16, options [nop,nop,TS val 1813568870 ecr 2309630659], length 0
02:53:24.706132 IP kali.36926 > 192.64.151.235.http: Flags [.] , ack 351, win 501, options [nop,nop,TS val 557792275 ecr 1817422025], length 0
02:53:24.729104 IP 192.64.151.235.http > kali.36926: Flags [.] , ack 331, win 507, options [nop,nop,TS val 1817432243 ecr 557781120], length 0
02:53:25.218761 IP kali.36080 > lcigaa-az-in-f3.1e100.net.http: Flags [.] , ack 1, win 508, options [nop,nop,TS val 1061541065 ecr 1681157381], length 0
02:53:25.248162 IP lcigaa-az-in-f3.1e100.net.http > kali.36080: Flags [.] , ack 1, win 1050, options [nop,nop,TS val 1681167532 ecr 1061510532], length 0
02:53:26.389609 IP c0:35:32:eb:8f:5b:49803 > 192.64.151.235.http: Flags [S.], seq 4106814254, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK], length 0
02:53:26.429730 IP 192.64.151.235.http > c0:35:32:eb:8f:5b:49803: Flags [S.], seq 1714568250, ack 4106814255, win 64240, options [mss 1460,nop,nop,sackOK,nop,wscale 7], length 0
02:53:26.429733 IP c0:35:32:eb:8f:5b:49803 > 192.64.151.235.http: Flags [.] , ack 1, win 1026, length 0
02:53:26.539477 IP c0:35:32:eb:8f:5b:49804 > 192.64.151.235.http: Flags [S.], seq 2217868314, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK], length 0
02:53:26.566003 IP 192.64.151.235.http > c0:35:32:eb:8f:5b:49804: Flags [S.], seq 3687990267, ack 2217868315, win 64240, options [mss 1460,nop,nop,sackOK,nop,wscale 7], length 0
02:53:26.568167 IP c0:35:32:eb:8f:5b:49804 > 192.64.151.235.http: Flags [.] , ack 1, win 1026, length 0

```

f- Capturer les paquets provenant d'une source exacte : tcpdump -i eth0 src @IP (pour cela tester avec un ping depuis kali vers XP ou vice versa)

```

(leyti002@kali)-[~]
$ sudo tcpdump -i eth0 src 192.168.2.25
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
03:06:35.558936 ARP, Request who-has DESKTOP-SQND070 tell kali, length 28
03:06:35.564802 IP kali > DESKTOP-SQND070: ICMP echo request, id 1, seq 1, length 64
03:06:35.683416 IP kali.60825 > _gateway.domain: 3447+ PTR? 174.2.168.192.in-addr.arpa. (44)
03:06:36.578653 IP kali > DESKTOP-SQND070: ICMP echo request, id 1, seq 2, length 64
03:06:37.596818 IP kali > DESKTOP-SQND070: ICMP echo request, id 1, seq 3, length 64
03:06:38.605137 IP kali > DESKTOP-SQND070: ICMP echo request, id 1, seq 4, length 64
03:06:39.614260 IP kali > DESKTOP-SQND070: ICMP echo request, id 1, seq 5, length 64
03:06:40.279365 ARP, Reply kali is-at 08:00:27:79:ec:b1 (oui Unknown), length 28
03:06:40.866051 ARP, Request who-has _gateway tell kali, length 28
03:06:40.966227 ARP, Reply kali is-at 08:00:27:79:ec:b1 (oui Unknown), length 28
03:06:44.745969 IP kali > DESKTOP-SQND070: ICMP echo reply, id 1, seq 1, length 40
03:06:45.853320 IP kali > DESKTOP-SQND070: ICMP echo reply, id 1, seq 2, length 40
03:06:46.950200 IP kali > DESKTOP-SQND070: ICMP echo reply, id 1, seq 3, length 40
03:06:48.008769 IP kali > DESKTOP-SQND070: ICMP echo reply, id 1, seq 4, length 40
^C
14 packets captured
14 packets received by filter
0 packets dropped by kernel

```

g- Capturer les paquets d'une destination exacte : tcpdump -i eth0 dst @IP (pour cela tester avec un ping depuis kali vers XP ou vice versa)

```

(leyti002@kali)-[~]
$ sudo tcpdump -i eth0 dst 192.168.2.174
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
3:16:01.711062 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49753: Flags [P.], seq 11137677:11137787, ack 3910210860, win 325, length 110
3:16:03.709388 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49843: Flags [P.], seq 3676785020:3676785130, ack 2228566027, win 302, length 110
3:16:03.749325 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49843: Flags [.] , ack 1, win 302, length 0
3:16:06.745508 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49753: Flags [P.], seq 110:220, ack 111, win 325, length 110
3:16:07.308026 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 1, length 64
3:16:08.383377 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 2, length 64
3:16:08.749709 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49843: Flags [P.], seq 110:220, ack 111, win 302, length 110
3:16:09.392232 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 3, length 64
3:16:10.404115 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 4, length 64
3:16:11.421225 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 5, length 64
3:16:11.832258 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49753: Flags [P.], seq 220:334, ack 111, win 325, length 114
3:16:11.839353 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49753: Flags [P.], seq 334:444, ack 221, win 325, length 110
3:16:11.965726 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49753: Flags [.] , ack 331, win 325, length 0
3:16:12.099954 ARP, Reply kali is-at 08:00:27:79:ec:b1 (oui Unknown), length 28
3:16:12.516474 ARP, Request who-has DESKTOP-SQND070 tell kali, length 28
3:16:12.519551 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 6, length 64
3:16:13.542284 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 7, length 64
3:16:13.883423 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49843: Flags [P.], seq 220:330, ack 221, win 302, length 110
3:16:14.589000 IP kali > DESKTOP-SQND070: ICMP echo request, id 3, seq 8, length 64
3:16:16.873024 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49753: Flags [P.], seq 444:554, ack 441, win 325, length 110
3:16:16.887455 IP Fibe-TV-Box-FibeTV.8009 > DESKTOP-SQND070.49753: Flags [.] , ack 441, win 325, length 0
^C
1 packets captured
1 packets received by filter
0 packets dropped by kernel

```

h- On peut aussi essayer de capturer les mots de passes et les noms d'utilisateurs envoyés en claire. Pour ce, taper la commande suivante :

```

...w...h...F..XK..y...#...Q...m/q...T...F...Q...jw...3...C...4...S...m...Q...w...
...uM-q...Y...Y...E...S...(\...w...S6...k*0...3...jXK.../..n.k.I...2...))...0...-...Or..x.v...x...m.egDYA..n8m...k...BE...t...
A...9g...n.k...n...k...e...*A...F...%...a...d...Z...l...A...e...3Z...1...jz.R.V...M.P.*...~*...t...A...A...?(...g...g@...|...z.Kd.nA..4...?...%M...
].P...T...3...6...Y...3...S...H...nt...a...8...VN...JM...4...o...3...Y...B...j...p...Qh...j...Y...U...k...Q...<...<...
4...-...V...3...6...Y...3...S...H...nt...a...8...VN...JM...4...o...3...Y...B...j...p...Qh...j...Y...U...k...Q...<...<...
...qQ...<1P...o.hZ...0...m...j.MQ0...i...a.M...g...3E...9...C...w...jE...f...w...i.Z...C...*...LB...f...Wh.Q...2.W7...:...%0...t.2...(^L...j...t.S.Cc...
2I...:3...<...1>...G...85...i...e...0...W...<...g...L...%...n...S...6.../...S...ak...1t...n...=...{...0...{MnaFtr^...n...>...UYNd...f...}]...%...Q...L...jz.7...;...{...
f(g...[.2...-...QIP...-...Q...i...P?7.R.K...t...>...D...>...2...=...x.02...?i...r(y...l...)]...0...6eh)...].PsF.@...
...D...-...y.F...L...u...0.S...g...kd.$X...w.H...3...q.h...J...|...>...T...+...[5...4...^k...fT...qLn...qLn...-H...PF.Q...?dLC$^.$1...sTgI...L>q...
I.X...e...y10...k...+6.../...t...h...1...$...b.m.v...j...>...W...Fi...2...12..."+...?...m.v0...dQ...Hu...r...<...=...>...B1.V1...36.N...BN...i2.8.0p...
N...S...Qc...+...8d...t...).6......7...2...w...C...n4.R...$.t1...*...c...^...?...$...6.I...m...m...H)...Tj;1...^...+...s?C...m.Z...
...-...J...4...A.Ci...}C#...d...0...4...C...
...SP5T...u...d...*...@j...A.y...m...B...p.D...2...;1.B%.c...d...}..LMqe.D+;..cw...o...LZs...r...L...T.D...Q...P...
'c.l"...Z7x.CP...Du...[...I...6...;...H.f...:...m...2q...k...p...=...?..D9x{a...^C...0...cd...uU!...>lv...rz...0...}#...-aA.3.F...1%...{ $r...|G...1C...A...!Qp...
'p...n...N...Ktu...f...f...>..cv..jWw...>...?...WxR.G...:...~[.7...);...;...\...t..."...UNP...-1...>...9...@sq.0...nwF...s.M...a%...I9/...X.er...
...e.n...IU'...D/...XM.v:7.0.*...A.C...A...=n0.8...y...Rd.r...{...y...>...t...D-Lxw%.Y>.)\...}3...~...-E...g...'.0...8<i7...q... 'E...p...D'...
...S...V+U...+...C...*...dU...Zz...*...Z...Q...aF...0...C...l...QP]...R...PI[bt+...o{...#...!p.t...%\...
...T...C...M'.M...W.W...r...Xv...".h...;3r.2.6%...}S...:6ap...=...J...?..2...I...$...-P...Z]nx...z...UO.l...d...q)...Cw...P...F...].%...
y...x...C...'.GEK...R...CN7H...3s0.$... \G.Q.r.y...OX#...}:a{...=...x...{...}W...o4..I.12]...zC'%{ xH...v.f.S...E...6...j...}.wK.y%xx...[-...D.Z...
.N-3.mg...D.a.U...6IP}\^...3BK<rw...q...q.4..W#_o...f.gAp...*...N...t...)|...cz...Y8Kw...M4...gce.W...q...lw.6...6...3F...U.b...f...F.W\<...M.Z...
E.B...f...f...R...i.WX...g...$.|...s.j...:...PB!..._#...;!"!X...=...PM...:lv5N...u...[...=...t.n.p.0.gN...5.kx%Q...6N[KL...{p.5.X.u...+...:...7...N...#...
...m.s...f...WB*...{...l...Q...Z...P...f...2+@h...H.S.#...i.k-"/...N...YB...R...u...?LGune...V...zZL...3c</...?...;...%...8G...LM...s...P.f...Q...
'8[e8f...i.w...+...A.s...#...8x)...=n...jg:...5.R...L...S...f).0...W.g.g.dkTe9..S/V...>...q.d... (4..T.B...e...$.Q0.Y.z<...T...2...H.eR...{...k...%QgJ...
A...r.l...<...+...R.i7...Z...!...t]J...Rc[...s...8...).yk.P.R...pc.H...g...I...[.B.[...8BM...).7...;...9...`y.8L1g...d...y~<...D...0...s...M...t...@...B%...
j.VO...rl...SYT.I...e...].e...mg...g...n...-?)...r...r...Z...Z...VG...[.lx.U.7.B.0;...s.z.4..5.4.X... (....:....D.k...=...6.0..."=L.l....[...M...{...<...
6.<K...U...v.m.a.0...Y...YD...hyj...><(\.r...*2.tf...96...r.J<T1h.P...f).T...Sz...(.0).d...{Q...D./456...>>8...j...G...N...+.../$...G...n...(\...n\...
/y...%FQZ.J.K.*.VPM=E...(.t1...@...z...0... \...z.../...<G.O.O... "PEc...]....p.o.X...bb0...7g...J'0...5%N!U...;t1...g...}...g7h...
...yp...T.Y...c...l...v...4...u...B...').T.6...I7...V...{...6...H.ZB...A...BuN...9...!C...T...!...1?...c...tW... 'O.w.L>9..6.W...A.%...
...v...#...K...Z.D...@*...K.X.5@w.w.R0.$3.Wb...-c...
...W...s...8...|...z...f...QF...AAK.G...g...J...#C...>...!...!!?...Jk...U...}.%...qx%...Y...LV...wf.h...>+a'4...U...w...).o...v.ZZ!Ty...
...i%...8...bc...}Q...{...U...".XG...Y...Y...~...-G...F.f.a.D...{...J...W^...L...4...>dQ...3...)}P...S...6...[3d...e...e...hH...V7...
...A...>...J.B...U...}[...LM.roN$.am...g...@J.S%...TV...#...Y...5...6...a-Nkp...-YK...^...r...*...7I...<'...fB...kal.s...8K.i.j.0...W...ow$!k#e9...r...3...
...B...U...tOp...K.k.s.../...k...Gg.W.=4... ( ...H...^...8...6D...o...? ...VH[g...7Y9...azR.o.v...*...2...*7...nU...c.G...tY...iI...j...d.l...r...dS...ez)...3...a...
...*.H...Z)z.9...{...@...G...).RZ...bO...$.D...0...3...kF...*YO.F.6t...3'.5?..o.c9...N...oH...j...C.K.%/Q...X.V.U.P*...D$CB... )...>...0...
R.G...Xw...@...QM..._On.D..._0.g@.6...6.s...n.72...w...@J.\bNeq>.#y...P.v.!/...3...*...Ud...9.^H...;...))x.xZ.0...VL.Z...@D...v...33j...
...b...f...X...a...$...^)...b.Z..._...1...j;A...wj$6...:...Y...!~s...'K.\K...c@G...j...l.2'...L... (f...c...-y.b...CCv...N...t**m...gs09D..._g5...t..._2...
...w... "fM..." +u...0.F...Rx...[...k...+...v...V[.7...
j!..Z...#Q...g...UN...w.z.C./n.bU...S...a...V...cT:d:x.g...f...i...:h...Eol.3.R.X.hVh...j..._...vbLQ/I...N...K...~...y@...@...uB.C.21...Ay...
username=admin&password=password&login=Login
username=admin&password=password&login=Login
^C13249 packets captured
16238 packets received by filter
2987 packets dropped by kernel

```

k- Documentez-vous sur les options de tcpdump, effectuez une recherche à quoi sert l'option -vvv et la commande egrep -i.

L'option **-vvv** (triple verbose) demande à tcpdump d'afficher le maximum de détails pour chaque paquet : TTL, longueur totale, checksum IP et TCP, options TCP, flags détaillés, numéros de séquence complets. Chaque niveau de v supplémentaire ajoute une couche de détail. Sans cette option, tcpdump affiche une sortie condensée qui omet beaucoup d'informations utiles à l'analyse.

La **commande egrep -i** est une version étendue de grep qui supporte les expressions régulières complexes. L'option -i rend la recherche insensible à la casse, donc PASSWORD=, Password= et password= sont tous détectés. Le symbole | entre les termes fonctionne comme un OU logique, permettant de chercher plusieurs patterns simultanément dans un seul pipeline.

I. Expliquez quelles informations sont visibles dans tcpdump et lesquelles sont plus détaillées dans Wireshark.

Tcpdump affiche en ligne de commande : adresses IP source/destination, ports, protocole, flags TCP, numéros de séquence, taille des paquets, et avec -vvv les TTL et checksums. Tout reste en texte brut dans le terminal.

Wireshark offre une interface graphique qui décompose automatiquement chaque couche du modèle OSI, reconstruit les flux TCP, décode les protocoles applicatifs (HTTP, DNS, ICMP) en champs nommés et cliquables, affiche le contenu hexadécimal et ASCII simultanément, et permet des filtres d'affichage très précis. La section HTML Form URL Encoded de la Tâche 3 n'est possible que dans **Wireshark** — **tcpdump** affiche le même contenu mais noyé dans du texte brut, comme visible dans **ta capture h**.

m. Quels sont les avantages et les limites de tcpdump dans un contexte de pentest ?

- **Avantages** : tcpdump fonctionne entièrement en ligne de commande, ce qui le rend utilisable sur des serveurs sans interface graphique ou via SSH. Il est léger, très rapide, et peut être combiné avec des outils Unix comme grep, awk ou sed pour filtrer et analyser en temps réel. Il peut écrire directement dans des fichiers. pcap lisibles ensuite par Wireshark.
- **Limites** : l'absence d'interface graphique rend l'analyse de protocoles complexes difficile. Il ne décode pas automatiquement les protocoles applicatifs comme Wireshark. L'analyse de gros volumes de trafic en texte brut est fastidieuse et nécessite une bonne maîtrise des expressions régulières.

n. Dans quel cas préféreriez-vous utiliser tcpdump plutôt que Wireshark ?

Tcpdump est préférable quand on travaille sur une machine distante accessible uniquement via SSH, quand on veut capturer rapidement du trafic sur un serveur de production sans installer une interface graphique, ou quand on souhaite automatiser la détection de patterns spécifiques dans un script. La commande de la section h en est un parfait exemple : en une seule ligne, on capture et filtre en temps réel les credentials sans aucune interface graphique.

o. En vous basant sur vos captures, expliquez comment vous pourriez identifier un comportement anormal uniquement avec tcpdump. Quels indicateurs dans la sortie pourraient suggérer :

- **un flood ICMP** : Dans une capture normale comme en f), les ICMP echo request arrivent à intervalles réguliers d'environ 1 seconde (seq 1, 2, 3...). Un flood ICMP se manifesterait par des dizaines ou centaines de lignes ICMP par seconde avec des timestamps quasi identiques, souvent depuis une même source vers une même destination. Le compteur final afficherait des milliers de paquets capturés en quelques secondes.
- **un scan** : Un scan de ports (nmap par exemple) produirait de nombreux paquets TCP avec uniquement le flag [S] (SYN) vers des ports différents d'une même machine cible, sans les ACK correspondants — c'est la signature d'un SYN scan. On verrait aussi des connexions vers des ports inhabituels en très peu de temps.
- **une activité inhabituelle** : Dans ta capture g), le trafic de Fibe-TV-Box vers Windows 10 sur des ports non standard (49753, 49843, 8009) mériterait investigation si ces machines n'étaient pas censées communiquer. Une activité inhabituelle se reconnaît par des connexions vers des IPs ou ports inconnus, des volumes de données anormaux, ou des protocoles inattendus pour une machine donnée.